Applied Energy 338 (2023) 120932

Contents lists available at ScienceDirect

Applied Energy

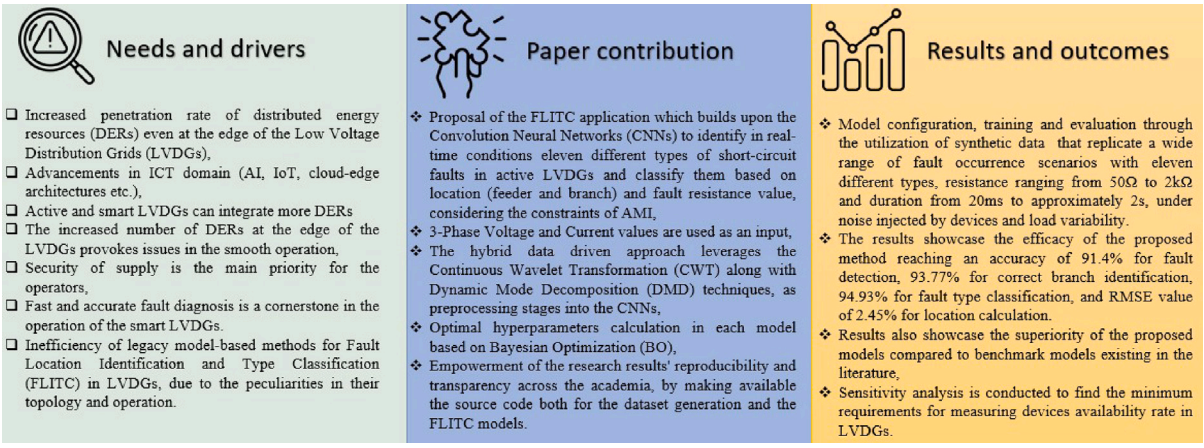
journal homepage: www.elsevier.com/locate/apenergy

Deep learning-based application for fault location identification and type classification in active distribution grids☆

V. Rizeakos, A. Bachoumis *, N. Andriopoulos, M. Birbas, A. Birbas

Department of Electrical and Computer Engineering, University of Patras, Rio Campus, 26504, Patras, Greece

GRAPHICAL ABSTRACT



ARTICLE INFO

Keywords:
Active distribution grids
CNNs
Deep learning
Fault detection and location identification
Wavelet transformation

ABSTRACT

The high penetration of distributed energy resources, especially weather-dependent sources, even at the edge of the distribution grids, has increased the power system uncertainties and drastically shifted the operational status quo for the system operators. For the operators to ensure the uninterrupted electricity supply of the end-consumers, the fast and accurate response to fault events is of critical importance. This paper proposes a data-driven fault location identification and types classification application based on the continuous wavelet transformation and convolutional neural networks optimally configured through Bayesian optimization. This application leverages the proliferation of high-resolution measurement devices in distribution networks. It can locate the exact position of the short-circuit faults and classify them into eleven different types. Its intrinsic models grasp the spatial characteristics and the converted in frequency domain temporal ones of the three-phase voltage and current timeseries measurements stemming from the field devices, thus increasing the operators' visibility of their networks in real-time. We conduct simulations through synthetic data, which we provide in an open-source repository, that replicate a wide range of fault occurrence scenarios with eleven different types, with the resistance ranging from 50 Ω to 2 kΩ and with duration from 20 ms to approximately 2 s, under noise conditions injected by devices and load variability. The results showcase the efficacy of the

☆ This research has been financed by the European Union's Horizon 2020 project "FEVER: Flexible Energy Production, Demand and Storage-based Virtual Power Plants for Electricity Markets and Resilient DSO Operation", under the grant agreement ID: 864537 in the H2020-LC-SC3-2018-2019-2020 call.

* Corresponding author.
E-mail addresses: up1053537@upatras.gr (V. Rizeakos), abachoumis@ece.upatras.gr (A. Bachoumis), nadriopoulos@ece.upatras.gr (N. Andriopoulos), mbirbas@ece.upatras.gr (M. Birbas), birbas@ece.upatras.gr (A. Birbas).

https://doi.org/10.1016/j.apenergy.2023.120932
Received 29 July 2022; Received in revised form 14 February 2023; Accepted 3 March 2023
Available online 11 March 2023
0306-2619/© 2023 Elsevier Ltd. All rights reserved.

proposed method reaching an accuracy of 91.4% for fault detection, 93.77% for correct branch identification, 94.93% for fault type classification, and RMSE value of 2.45% for location calculation.

1. Introduction

In the ever-evolving environment of the active distribution networks, the uninterrupted and high-quality supply of the end-customer is threatened due to the intermittent nature of the Renewable Energy Resources (RES). Grid operators shall ensure power system reliability by performing high grid observability even at the edge of the Low-Voltage Distribution Grid (LVDG) in real-time conditions to achieve rapid recovery after the emergence of contingencies. The advancements in Information and Communication Technology (ICT) through the emergence of low-latency telecommunication networks and Advanced Metering Infrastructure (AMI) [1], along with the accelerated progress in Machine Learning (ML) and especially in Deep Neural Networks (DNNs) [2], can act as a catalyst for alleviating the problems arising in rich distribution networks, i.e., highly RES-penetrated with a significant number of prosumers [3,4].

Active and smart LVDGs can self-heal in case a fault occurs. Several strategies are followed by the grid operators to react to contingencies and apply self-healing control practices [5–8]. The cornerstone of these practices is the execution of accurate and fast actions for fault diagnosis, i.e., fault detection and identification of the specific location and type. However, this activity is not trivial in LVDGs, due to particular characteristics that impede the traditionally used methods. Specifically, LVDGs have a high number of branches, are multi-phase, and usually have an unbalanced operation due to single-phase connected loads. Different types of conductors do exist, connecting the nodes with different characteristics and lengths, leading to a wide range of resistance (R) and reactance (X) values with a high R/X ratio. In addition, a limited number of AMI devices exist, reducing the overall observability, and have a radial structure and operation [9]. Therefore, the development of a fault diagnosis method for LVDGs shall consider from the initial design process all the above-mentioned inherent characteristics of the LVDGs.

1.1. Literature

The approaches used for fault diagnosis in power systems can be classified into three categories. The first category includes the classical approaches that use direct modeling techniques, with the most important being the impedance-based [10–12] and traveling-wave methods [13,14]. System operators have widely used over the last decades these two methods to perform fault diagnosis in high and medium voltage networks and are dependent on the line parameters. However, for both methods, their efficiency in LVDGs decreases significantly. For the former, the ability to accurately detect the fault location deteriorates due to the high number of branches at the LVDGs [15]. The latter method lacks accuracy in LVDGs, due to the presence of several branches, impending the differentiation between waves [16]. The exact fault location is crucial to be identified in rich LVDGs to ensure the security of supply for the end-consumers.

The second category consists of data-driven methods with a wide spectrum of techniques from signal processing to Artificial Intelligence (AI) domains. In both [17,18], fault diagnosis methods using Support Vector Machines (SVMs) in a real distribution feeders are presented. [19] proposes a fault diagnosis method in LVDGs based on gradient boosting trees. Moreover, several works with applications both at the transmission and distribution levels leverage domain transformation-based methods, such as wavelet transform, to conduct fault detection [20–22]. In [23], a hybrid clustering algorithm based on k-Nearest Neighbors (k-NN) and k-Means is developed, using as a preprocessing method the Matching Pursuit Decomposition (MPD). A complementary clustering technique, the Density-Based Spatial Clustering of Applications with Noise (DBSCAN), has been implemented in [24] for fault diagnosis. The feature selection and dimensionality reduction methods of Principal Component Analysis (PCA) and Random Forest (RF) have been used for active LVDGs in [25,26], respectively. A Multi-Layer Perceptor (MLP) along with Extreme Learning Machine (ELM) are deployed in [27], particularly for radial grid topologies. DNNs are employed in [28] for identifying the fault location and type for radial topologies in LVDGs. Finally, Convolutional Neural Networks (CNNs) have also been leveraged for fault diagnosis in both transmission and distribution levels [29,30].

The last category includes methods that use a hybrid approach for fault diagnosis. These methods combine data-driven and model-based approaches to conduct fault location and type identification processes. In [31], the impedance-based method and DNNs are employed, where in [32], a ruled-based fuzzy logic is developed, which can detect the difference between simulated data and the actual measurements to extract the accurate short-circuit fault location.

In the data-driven methods, the high non-linearity of the fault detection problem is approximated in a sufficient way, thus overpassing the limitations that the model-driven techniques face in the distribution grids. However, a large amount of data is required for the sufficient training of data-driven methods, while covering all the potential faulty scenarios. Otherwise, the data-driven method will perform inadequately.

1.2. Work contributions

From the above-conducted literature review, we can conclude that in the context of LVDGs, data-driven methods have gained popularity mainly for two reasons: (i) the limitations that model-driven methods experience at the edge of the grid and (ii) the accelerating integration of AMI even at the residential end-consumer level. For the former, already published works demonstrate the superiority of the data-driven methods compared to the model-driven methods at the edge of the distribution grids [33]. For the later, by proposing methods that provide added value to the operators through the digestion and analysis of high-resolution data stemming from the AMI, more technical benefits of the AMI integration could be justified and considered into the Cost Benefit Analysis procedure, thus accelerating the deployment of advanced measuring devices at the edge of the distribution grids.

However, the amount of data-driven works that focus on the LVDGs is limited [33]. In addition, the existing works have been tested in non-open-source datasets, thus direct comparison of the methods' performance is not feasible. To fill this gap, this work proposes an open-source application for smart, rich, and radial LVDGs for Fault Location Identification and Type Classification, called hereafter the FLITC application, based on advanced DNN architectures. Specifically, Continuous Wavelet Transformation (CWT) and CNN architectures are employed to consider the spatio-temporal characteristics of the AMI measurements. This work aims to detect the fault occurrence, its type, and exact location in real-time conditions. This information is used as an input into an advanced self-healing application aiming at repairing the contingencies and performing energy restoration efforts to reduce the impact of energy interruption on the consumers, by decreasing the number (SAIFI) and duration (SAIDI) of the interruptions [34].

The main contributions of this work can be concisely described as:

- Propose of an application of DNNs to identify in real-time conditions eleven different types of short-circuit faults in active LVDGs and find the exact location (feeder and branch) and distance from the root node, considering the constraints of AMI devices resolution and the computational time,

- Propose an application that replaces the existing protective schemes in the distribution grids, by leveraging the high-resolution data stemming from the AMI devices,
- Employ both the CWT along with Dynamic Mode Decomposition (DMD) techniques as preprocessing stages into the CNNs, constituting the application's cornerstone. To the authors' knowledge, it is the first time across literature that this data-driven approach, empowered by methods of the signal processing domain, is used for the fault detection and location identification application in active LVDGs, under noisy measurements that are introduced by the AMI and the variability of loads and distributed generation,
- Generate dataset that simulates the LVDG operation under fault occurrences for tuning and evaluation of the application,
- Tune hyperparameters optimally in each model using the Bayesian Optimization (BO) algorithm and particularly the Tree Parzen Estimator (TPE),
- Showcase the superiority of the proposed models compared to benchmark models existing in the literature, and
- Empower the research results' reproducibility and transparency across the academia, by making available the source code both for the dataset generation and the FLITC application, in an open-source repository [35].

1.3. Paper outline

The paper is structured as follows: Section 2 includes a thorough description of the problem. Section 3 presents the mathematical building blocks and concepts upon which FLITC application is constructed. Section 4 provides the proposed FLITC application, a deep analysis of the DNN-based architecture, the algorithm used for hyperparameter tuning, and the employed loss functions. Section 5 includes a description of the use case serving for validation purposes. Section 6 introduces the extensive exploration of the proposed application's results to showcase its efficiency and applicability. We conclude our work in Section 7, where a summary of the proposed application and recommendations for further work are given.

2. Problem statement and specifications

This section includes the description of the fault detection problem and the minimum requirements the FLITC application has to meet to perform with high accuracy. Initially, a description of the faults considered in this work and their physical meaning takes place. Then, the specifications of the AMI and the corresponding reporting rate are presented. Finally, based on this analysis, we derive the functionalities of the FLITC application.

2.1. Fault types

As reviewed in [36], failures in electrical grids are categorized based on their origin. Natural-made causes are the most common type of occurring faults in power grids and mainly comprise disturbances due to extreme weather phenomena. Another category of faults emerges from malfunctions in electric grid equipment, as well as from human failure. Lastly, a portion of the occurring faults is associated to man-made hazards, i.e., either cyber-attacks that aim to affect the grid integrity or other forms of intentional attacks.

Based on the fault origin, its severity has a wide range. On the one hand, a lightning strike can cause instability in the power flow of the electric grid, while on the other hand, it can create a localized blackout due to the destruction of the grid infrastructure. The aftermath of a fault occurring to the grid might overlap with two or more of the possible causes discussed above, e.g., a localized blackout might stem from both a lightning strike and a substation malfunction. This work investigates a subset of all the possible types of errors. More specifically, the proposed methodology emphasizes the detection of single-phase (1Φ), double-phase (2Φ), and three-phase (3Φ) short-circuits with respect to ground and phase-to-phase ones. According to [37], the origin of those faults is mainly attributed to physical contact between one or more phases with the ground (i.e., tree fall, broken insulators, natural phenomena such as lightning storms, hurricanes, floods, heatwaves, etc.), overloading, corrosion or lack of maintenance of solar plants and wind turbines, generators' overheating, short-circuit of generator rotor windings, etc.

Table 1
Occurring frequency and severity of different line fault types.

Types	Frequency	Severity
L-G	70%	Low
L-L	7.5%	Medium
L-L-G	17.5%	Medium
L-L-L	2%	High
L-L-L-G	3%	High

The duration of short-circuits in distribution grids can vary depending on several factors, including the specific system and the type of short-circuit that occurs. In general, short-circuits in distribution grids can last from a few milliseconds to several seconds, with the majority lasting less than one second [38]. This is because most distribution systems have protection devices, such as circuit breakers, that are designed to quickly detect and isolate the faulted section of the grid to prevent damage and minimize disruption.

The frequency of occurrence of each short-circuit fault depends on its type. According to [39], the most common type is the 1Φ short-circuits with the ground at a rate of 70%, while around 5% concerns 3Φ short-circuits. The remaining 25% includes phase-to-phase short-circuits, and double-phase short-circuits with the ground. For the purposes of this work, it is permissible to consider that the respective frequency for each of the five types of fault is as follows: 70% for 1Φ to ground, 17.5% for 2Φ to ground, 7.5% for line-to-line, 3% for 3Φ to ground, and 2% for 3Φ fault. When one or two lines contribute to short-circuits, this creates a significant imbalance in a most likely already unsymmetrical power grid, where each phase's power consumption differs. However, this effect restricts the power flow of the affected lines when more are involved. Thus, it is acceptable to assume that the severity of a fault increases with each contributing phase (Table 1).

2.2. Metering infrastructure

AMI enables the collection of data stemming from the grid's buses, such as 3Φ voltage (V_{RMS}), 3Φ current (I_{RMS}), henceforth denoted as $3\Phi - V$ and $3\Phi - I$, respectively, and active power (P), and transmits them to the operators' data management system. Since most of the faults' duration does not exceed 1 s, the application shall be able to detect faults ranging from a cycle, i.e., from 20 ms up to 2 s. In addition to that, since data arriving at such a high frequency will be difficult to be handled, it is considered that the data are sent in packages, e.g., every 5 or 20 s. Therefore, we consider that the application receives the data from each measurement with a 20 ms rate, and then accumulates the information in either 5 s or 20 s intervals. Data with high granularity can be provided by measuring devices, such as either Phasor Measurement Units (PMUs) or Power Quality Meters (PQMs), which have a sampling rate of up to 2880 and 100.000 samples/s, respectively.

2.3. FLITC specifications

Based on the above-described problem statement, the following requirements and specifications of the FLITC application are derived:

- *Measuring devices and variables*: The application uses $3\Phi - V$ and -I measurements from advanced measuring devices, i.e., PMUs and PQMs. I measurements are necessary at the root of each

feeder, while V measurements are necessary at least at the first and last node of each branch. In our model, we consider that we have V measurements from each node; an experimentation of the application's efficiency in different availability rates of V measurements takes place in Section 6.3. We assume that the measurements are synchronized.

- *Data handling and decision timeframe*: Because data arriving in high frequency are difficult to be handled, it is considered that the application shall be capable of receiving and analyzing data in packages, e.g., every 5 s or 20 s. Then, a decision (model inference) is taken in real-time conditions (sub-minute or sub-second scale based on the dimensions of the grid topology).
- *Faulty feeder detection:* The model has the capability of detecting the feeder in which the fault took place and differentiating that feeder from the healthy ones,
- *Faulty branch detection:* The model has the capability of accurately detecting which branch is the one that the fault has emerged from,
- *Faulty class identification:* The model can identify the type of the short-circuit fault that occurred in the LVDG. Eleven faulty classes are included: A-G, B-G, C-G, A-B, B-C, A-C, A-B-G, B-C-G, A-C-G, A-B-C, A-B-C-G, where A, B, and C denote the corresponding phases and G the ground,
- *Fault branch location*: The model can calculate the distance from the root node.

3. Technical preliminaries

This section presents the theoretical framework that the application is built upon. Initially, the CNN is introduced, which is the employed DNN architecture and is adept at capturing spatial changes in the input data. They are commonly used in image and video recognition algorithms and have further variations depending on the specific task. Then, the Dynamic Mode Decomposition (DMD) method is described, i.e., a completely data-driven dimensionality reduction technique independent of the underlying dynamics, firstly proposed in the hydrodynamics domain. Finally, the CWT method is presented that is used in the data preprocessing stage for data transformation from the time to the frequency domain.

3.1. Convolutional neural networks

CNNs are a widely used architecture of DNNs, which are primarily used in image and video recognition due to their advantage of determining spatial features in the given data. CNNs are feed-forward neural networks whose primary feature is the convolution of the input data with a shared weight kernel. The kernels slide across the input data and extract feature matrices for each convolution. Depending on the datasets' dimensions, the CNN's kernels or filters have a corresponding dimension. At the same time, the output of their matrix multiplications produces a feature map with lower dimensions through employing pooling. This process of consecutive convolution and pooling layers can continue to the point of creating a vector layer, also known as a dense layer. The input of a CNN model has dimensions of the following structure:

Dataset Length × Input Height × Input Width × Input Channels

In the case of image classification, for example, the dimensions correspond to the size of the dataset fed into the network, the picture height in pixels, the picture width in pixels, and the three-color channels (R, G, B) respectively. Then, a kernel with dimensions smaller than the above slides through the input and carries out matrix multiplication. The hyperparameters of kernel size, number of filters, stride, and padding of the kernel are all set by the user, according to the nature of the dataset. The input of the following stage is the feature map extracted from the previous one. This is passed through a pooling layer, which reduces the dimensionality of the processed data, either by using max or average pooling, which yields the kernel's maximum or average value, respectively. The flattened data from the pooling layer are fed into dense layers with a depth size determined by the user.

The selection of the CNNs in this work is based on two factors: (i) the CNN layer can capture the spatial information inherently included in the measured 3Φ timeseries data stemming from different locations of the LVDG, and (ii) the dimensions of the grid's branch data are similar to the CNN model input dimensions. Hence, the CNN can classify measured V timeseries data to a particular fault type by extracting data from the grid's 3Φ.

3.2. Dynamic mode decomposition

A dimensionality reduction technique that computes a set of modes, where each of them has a fixed oscillation frequency and a decay/growth rate [40]. These modes and frequencies are analogous to the normal modes of the system, but more generally, they are approximations of the modes and eigenvalues of the composition operator. Due to the intrinsic temporal behaviors associated with each mode, DMD differs from dimensionality reduction methods such as PCA, which computes orthogonal modes that lack predetermined temporal behaviors. Because its modes are not orthogonal, DMD-based representations can be less parsimonious than those generated by PCA. However, they can also be more physically meaningful because each mode is associated with a damped sinusoidal behavior in time.

Given a time series data X of size N-by-T where N is the number of variables and T is the number of time steps, then in any time step t, the first-order vector autoregression takes the form:

$$x_t = Ax_{t-1} + \epsilon_t \tag{1}$$

where x_t denotes the snapshot vector in time t and the size is $Nx1$, A is the coefficient matrix size of NxN, and ϵ_t is the error term. To find a well-behaved coefficient matrix and use it to represent temporal correlations, we reformulate the above equation as:

$$X_2 \approx AX_1 \tag{2}$$

Then, we employ the singular value decomposition of the X_1 to factorize it, writing:

$$X_1 = U\Sigma V^T \tag{3}$$

where U is consists of left singular vectors, V consists of right singular vectors and Σ is diagonal. We define $\widetilde{A}$ as:

$$\widetilde{A} = U^T X_2 V \Sigma^{-1} \tag{4}$$

The eigenvalues and eigenvectors of the $\widetilde{A}$ are then computed from the following equation:

$$\widetilde{A}y = \Lambda y \tag{5}$$

Finally, the DMD mode decomposition of the DMD eigenvalue Λ is given by:

$$\Phi = Uy \tag{6}$$

This work uses the DMD method to reduce the dimensions of the input dataset of the CNNs to facilitate the training and inference of the models by decreasing the requirements both for computational power and memory without significantly affecting the model's accuracy.

3.3. Continuous Wavelet Transformation

The CWT of a signal $s(t)$, can be defined as:

$$Z(\alpha, \beta) = \frac{1}{\sqrt{\alpha}} \int_{-\infty}^{\infty} s(t)\Psi^*(\frac{t-\beta}{\alpha})dt \tag{7}$$

where $s(t)$ is a finite energy signal, Ψ^* is the complex conjugate of the mother wavelet, and a and b are the scaling and translational factors

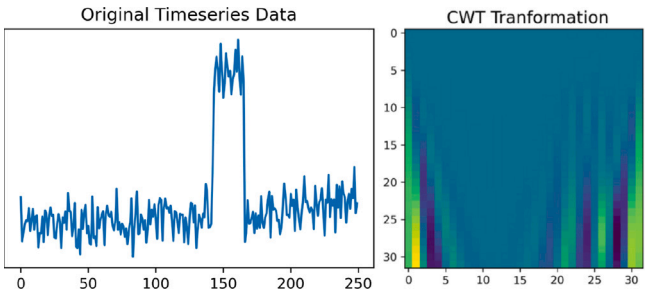


Fig. 1. Indicative example of CWT transformation for Vrms timeseries.

of the wavelet, respectively. Large scale values expand the wavelet in time, revealing low-frequency information in the signal, while smaller scale values shrink the wavelet and reveal high frequencies present in the signal. The CWT is calculated by continuously varying variables a and b over the range of scales and length of the signal, respectively.

Across the academia over the last years, many works provide solutions to classification and regression problems, using CWT as a preprocessing stage of the CNN models, especially for applications in the medical engineering domain [41,42]. In this work, the advantage of this approach is to transform the noisy $3\Phi-V$ and -I timeseries signals into the frequency domain and thus decompose complex information and patterns into elementary forms, thus boosting the ability of the CNN models to capture the faulty states/occurrences. This work uses Morlet as a mother wavelet, where an indicative example of this CWT conversion is illustrated in Fig. 1.

4. FLITC application

4.1. Architecture

The FLITC application fulfills the following functionalities:

- *Faulty feeder detection* through the employment of a feeder detection CNN model (FFNN),
- *Faulty branch detection* through the employment of a branch detection CNN model (FBNN),
- *Faulty type classification* in eleven fault classes, as defined in Section 2.3, through the employment of a faulty class detection CNN model (FCNN),
- *Faulty distance calculation* by estimating the distance from the root node through the employment of a distance calculation CNN model (FDNN).

The FLITC application's architecture is illustrated in Fig. 2. The developed components for the application's materialization are input, preprocessing, fault diagnosis, and output.

4.1.1. Input stage

It handles the measurements stemming from the AMI devices in the LVDG. Data are received in 20 s intervals in order not to obstruct the AMI communication system and create bandwidth issues. The accumulated 20 s data, i.e., feeder $3\Phi-I$ measurements, and each node $3\Phi-V$ measurements are segmented into four 5s batches, which are stored until all the timeseries have been processed. The data are successively forwarded to the next stage, responsible for the data preprocessing.

4.1.2. Preprocessing stage

The data preprocessing stage is an intermediate function between the incoming data and the diagnostic tools. It preprocesses the data as:

- *Cleaning step*: Checks the data for any missing measurements and replaces them based on their position in the input timeseries,
- *Grouping step*: Separates the I measurements from the V measurements and isolates the latter depending on the branch that each node belongs to,
- *Interpolation step*: As firstly introduced in [28], this step interpolates the measurements of each node to create a branch with a generalized number of virtual nodes, thus facilitating the distance estimation functionality by reducing the input vector. These virtual nodes are located in 5 different locations of each branch, i.e., 0%, 25%, 50%, 75%, 100% of the total faulty branch distance,
- *Normalization step*: Normalizes the data to insert them in the different models,
- *DMD step*: Reduces the dimensionality of the input data to alleviate problems provoked by memory overloading during the hyperparameter tuning and training of the models without significantly deteriorating the models' efficacy. This work uses DMD for reduction of the $3\Phi-V$ timeseries dimensionality,
- *CWT step*: Transforms the data from the time domain to the frequency domain to enhance the performance of the models. Even though the result of this process is a 32×32 image, the intermediate steps can vary. Therefore, two different CWT approaches are explored: (i) the CWT of the timeseries is a 32×250 image, retaining the original time dimension while extracting the data for the convolution via the mother wavelet to a new dimension with equal size to the final one, i.e., 32, (ii) CWT creates a 250×250 image, expanding the first dimension to get an orthogonal matrix including further information of the original timeseries. Each approach is used for different datasets according to the volume of information each CNN model requires. In this work, all the obtained best models (see Section 6) yield better results by employing the first approach, since the second one provides excessive information overwhelming the CNN layers.

Figs. 3 and 4 illustrate the preprocessing stage for the $3\Phi-I$ and $3\Phi-V$ input datasets, respectively, for the specific use case presented below in Section 5.

4.1.3. Fault diagnosis stage

The DNN blocks are the main computational blocks of the system since this is where the fault diagnosis of the LVDG is conducted. Data insert into each module as follows:

- FFNN module is fed with the preprocessed $3\Phi-I$ measurements,
- FBNN module is fed with the preprocessed $3\Phi-V$ measurements of the feeder classified as faulty in the previous stage,
- FCNN module is fed with the preprocessed $3\Phi-V$ measurements of the branch classified as faulty in the previous stage,
- FDNN module is fed with the preprocessed $3\Phi-V$ measurements of the branch classified as faulty in the previous stage.

The FFNN module receives the preprocessed input data and sends its classification results to the output. If the model predicts that the state of the LVDG during these 5 s is healthy, the fault diagnosis is concluded. Otherwise, if the model finds a faulty feeder, the process continues. Since the number of branches for each feeder can vary, the input shape of the FBNN is not standard. Thus, within the DNN training stage, models with feeders of different branches are trained. Therefore, the number of the DNNs selected from the training stage equals the number of different branch-size feeders. Depending on which is the faulty feeder, the corresponding fault branch is fed with the V data. The last stage involves both the FCNN and FDNN since they are independent and their input data have already been preprocessed.

Categorical cross-entropy loss function is used in the case of multiclass classification problems. It is a $softmax$ activation function (Eq. (8)), followed by a cross-entropy loss (Eq. (9)).

$$f(s)_i = \frac{e^{s_i}}{\sum_j^C e^{s_j}} \tag{8}$$

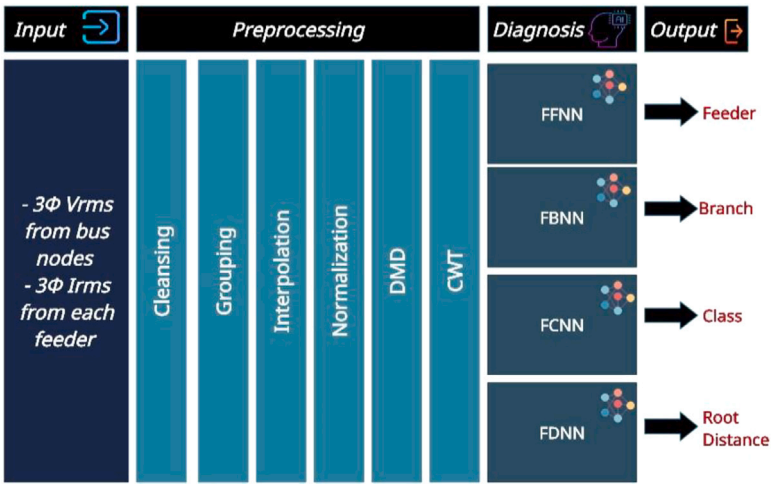


Fig. 2. Architecture diagram of the FLITC application.

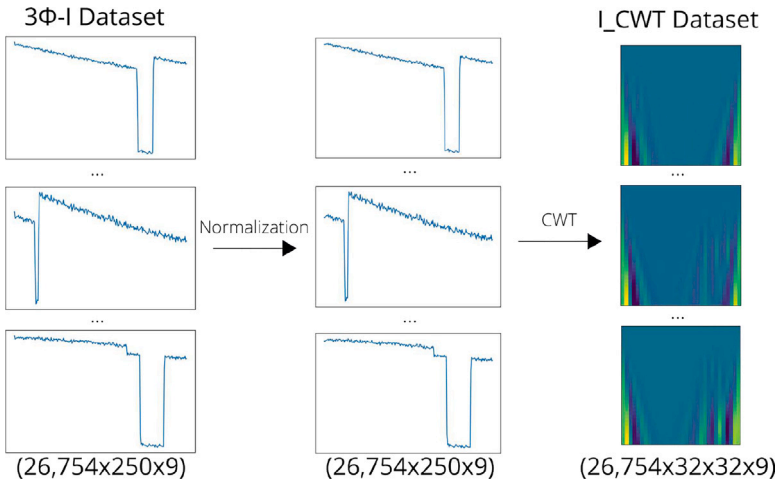


Fig. 3. Dimensionality (in parenthesis) and CWT transformation of the $3\Phi - I$ dataset during the preprocessing stage.

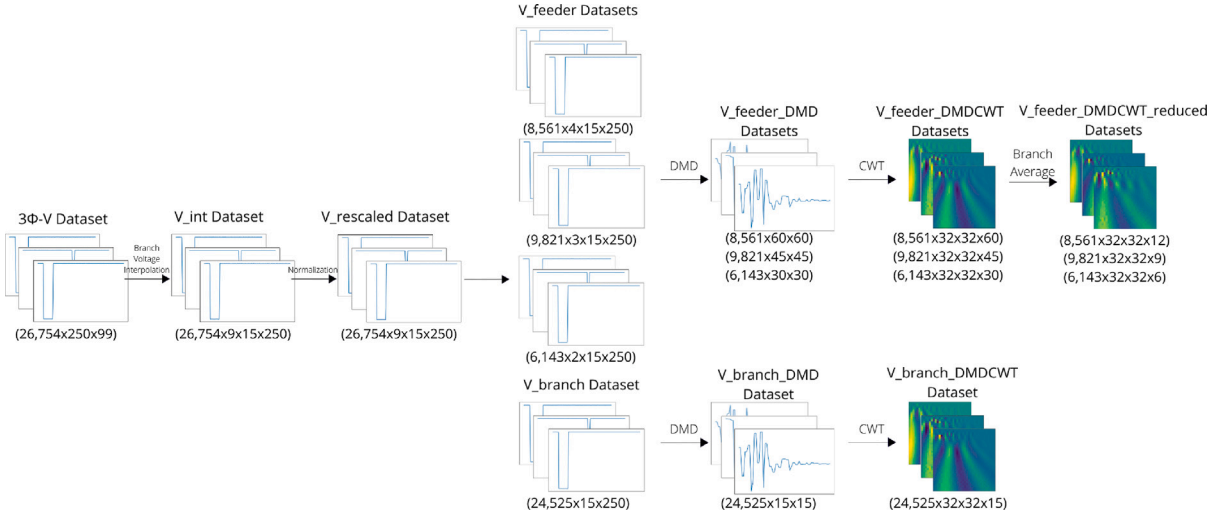


Fig. 4. Dimensionality (in parenthesis) and transformations of the $3\Phi - V$ dataset during the preprocessing stage for the FFNN (upper) and the FBNN (lower).

$$CE = -\sum_{i}^{C} t_i log(f(s)_i) \tag{9}$$

Since the labels are in one-hot encoding, only the positive class C_p keeps its term in the loss, so there is only one element of the target vector t which is not zero, i.e., $t_i = t_p$. Hence, by discarding the zero elements in the summation, the formula of the categorical cross-entropy is derived as such:

$$CCE = -log(\frac{e^{s_p}}{\sum_{j}^{C} e^{s_j}}) \tag{10}$$

4.1.4. Output stage

The DNNs output is provided in a comprehensive format and informs the user about the LVDG state. Specifically, it concatenates the results of all the four DNNs as follows: (i) *"No Fault Detected"* when the model classifies the LV grid as healthy, and (ii) *"C Fault Detected in (F, B, D)"* where C, F, B, and D denote the faulty class, faulty feeder, faulty branch and the estimated distance from the root node.

4.2. Modes of operation

Three different modes of operation can be defined for the FLITC application. Specifically:

4.2.1. Configuration and hyperparameter tuning

The configuration mode concerns the parameters configuration, such as the input vector dimensions of each DNN model and the number of the CNN models that will be applied for the branch detection. This mode is executed whenever the FLITC application uses a new LVDG. The required data for the configuration are the amount of LVDG feeders, branches per feeder, the amount and location of the devices that measure the $3\Phi - V$ values, and the length of each branch.

After the models' configuration, the hyperparameter tuning process is carried out. Hyperparameter tuning is the problem of optimizing a loss function over a graph-structured configuration space and thus calculating the optimal value of the model hyperparameters, i.e., the parameters that define the model's architecture and control the learning process. Computing the optimal hyperparameter value enhances the overall model's performance. As described thoroughly in Section 3, this work uses the CNN architecture as a foundation to perform the different tasks. The hyperparameters that need to be selected in this case are the batch size, number of hidden layers and units in each of them, the dropout rate for each layer, the window size in the incoming data, and the filter size.

However, this multi-dimensional hyperparameter space renders the methods used to identify the optimal values, such as the random or grid search, inefficient and time-consuming. To tackle this issue, the concept of BO has been introduced, where the number of samples drawn from the hyperparameter search space is probabilistically guided and reduced, thus allowing for proper evaluations of the most promising candidates for hyperparameters selection [43]. In this work, an automated approach for hyperparameter tuning of each model is performed based on the BO, namely, the Tree-structured Parzen Estimator (TPE) method [44]. The TPE is a sequential model-based optimization (SMBO) approach, which sequentially constructs models to approximate the performance of hyperparameters based on historical measurements. It then selects new hyperparameters to test based on this model. This method solves the problem of dealing with categorical and conditional parameters and therefore increases the efficiency of the hyperparameters selection process [44]. In literature, the TPE algorithm is broadly used in different domains and applications, such as image processing [45,46], load forecasting [47–49], and solar irradiance forecasting [50].

The TPE algorithm models $p(\theta|y)$ transforming that generative process by replacing distributions of the configuration prior with non-parametric densities [44]. By using in each iteration t different observations $(\theta^{(1)}, \theta^{(2)}, \ldots, \theta^{(M)})$ in the non-parametric densities, a learning algorithm is generated that can produce a variety of densities over the configuration space Θ. The TPE defines $p(\theta|y)$ such as:

$$p(\theta|y) = \begin{cases} k(\theta), & \text{if} \quad y < y^*. \\ l(\theta), & \text{if} \quad y \geq y^*. \end{cases} \tag{11}$$

where $k(\theta)$ is the density formed by using observations $\theta^{(i)}$, so that the corresponding loss $y = f(\theta^{(i)})$ is less than y^* and $l(\theta)$ is the density formed by the rest observations. The TPE algorithm chooses y^* to be some quantile γ of the observed y values, so that $p(y < y^*) = \gamma$, but no specific model for $p(y)$ is necessary. An Expected Improvement (EI) is defined as the fraction of the probability $k(\theta)$ divided by $l(\theta)$, and is maximized in order to select the $\theta^*(t+1)$ set for the next iteration. The algorithm terminates when the maximum number of iterations is completed. The runtime of each iteration of the TPE algorithm can scale linearly in $|H|$, i.e., the sorted history lists of observed variables, and linearly in the number of hyperparameters (dimensions) being optimized [44].

4.2.2. Training and testing modes

After the optimal configuration of the parameters mentioned above, training is conducted using the stochastic gradient descent method, specifically the ADAM optimizer, to calculate the models' weights and calibrate them on the specific network's topology and amount of metering devices. After the completion of the training process, the models' performance is evaluated throughout sample data. If the FLITC applications' performance is acceptable to the operator, the application can be used in a real operational environment.

4.2.3. Operational mode

During the real operational mode, the FLITC application can be hosted either in a server of the operator's control center or in an edge server located near the LVDG, e.g., the LV transformer, which has adequate computational resources to execute the application. The first implementation has the advantage of the computing capabilities of the cloud servers. However, high data bandwidth is needed for the data transmission to the server with the issues of data drop and security to exist. On the contrary, the edge implementation secures the LVDG data from breaches. It requires less transmission bandwidth because only the output is sent to the control center without transmitting the LVDG raw data. Regardless of the hosting environment, the FLITC application can be integrated into a self-healing application, offering real-time fault diagnosis and thus allowing for initiating other grid control functionalities in the LVDG for fault clearance and restoration of the consumer's power supply.

5. Use case

5.1. Simulated dataset

The dataset used for the CNN models' hyperparameter tuning, training, and evaluation process is generated in a simulation environment. For the dataset generation process, $1min-$ resolution real data of local consumption and distributed production are retrieved from the publicly available dataset of Pecastreet Dataport [51]. Fig. 5 illustrates the aggregated profile of the user's data. The penetration rate of the renewables for the case study is equal to 25%. Through the simulations, data interpolation is conducted to replicate the operation of a real LVDG and investigate a diverse and large number of fault scenarios, which are absent on already-open source available datasets that comprise on-field real measurements. Therefore, the simulated dataset can provide adequate information containing all the possible short-circuit-related faults that can occur in an LVDG. Furthermore, the generated data also include noise introduced by the measuring devices (Gaussian distribution with mean $m = 0$ and variance $s^2 = 10^{-4}$) and the load and generation variability; replicating the operation of a common unbalanced, radial high-RES penetrated active LVDG.

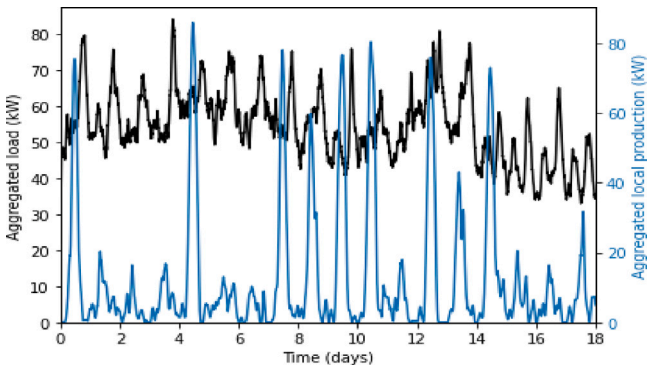


Fig. 5. Aggregated local consumption and generation.

The original LVDG, upon which this simulation model is based, is a Portuguese one given in [52] and shown in Fig. 6. A radial LVDG encompasses multiple feeders and secondary branches starting from an MV grid equivalent. Each node of the described network is integrated with a device that measures $3\Phi - V$, while the main feeders of the network monitor both the $3\Phi - V$ and -I measurements. A variable length distribution line is added between each node, emulating distribution losses over LVDGs. Furthermore, the consumer loads consist of variable parallel-connected resistive and inductive loads. Lastly, local distributed generation is simulated as a PV node to regulate the output of a PV system by adjusting the voltage of the system [53]. In case of a fault occurrence, this type of control can prevent the voltage from becoming too high or too low, which can cause power quality issues. Since our goal is to examine the applicability of our method to distribution grids as a complementary input to the existing protection schemes through the utilization of measurements from the AMI, in our study we considered that the faulty data used for the algorithms training/validation/testing have been impacted by the activation of the existing protection schemes (besides the high impedance faults that may bypass the protection schemes). In addition, we consider that the simulated PV inverters do not disconnect after the fault occurrence and the current contribution to the peak of the first cycle fault is about 20% of the aggregated PV inverter-rated current [54].

The flowchart for data generation, illustrated in Fig. 7. It uses as input the fault characteristics of each simulation scenario, as well as the consumers' loads and the local production of the distributed generation. The consumers' reactive power loads Q are calculated using a power coefficient equal to $\arccos\phi = 0.95$. In this model, during the fault event of each scenario, the resistance R between the 3Φ and the ground is drawn from a Log-Normal distribution, with an average value ranging from 50 Ω to 2 kΩ. Fault duration is also calculated using a Weibull distribution, with a duration ranging from 20 ms to approximately 2 s. Fig. 8 illustrates the duration and resistance histograms of the generated faults. It is noteworthy that the fault initialization is a uniformly distributed variable among the length of the simulation. In total, we generate a dataset with 26,754 samples, where 70% of it is used for training and 30% for testing purposes. The 20% of the training dataset is used for the validation of the model.

5.2. Baseline models

For the validation of our method, we are comparing it with the state-of-the-art data-driven models in the fault diagnosis and anomaly detection domains, especially dealing with timeseries data, which are:

- LSTM Autoencoder for unsupervised learning for detection of the outliers, i.e., the short-circuit faults in our case. This technique is utilized in several domains for anomaly detection, such as supply chain management [55] or network [56], where the model learns to forecast the signal of healthy states, then reconstructs the training data and calculates the Mean Absolute Error (MAE) for each training sample. The maximum MAE value is considered a threshold. If the reconstruction loss for a test sample is greater than this threshold value, then we can infer that the model is seeing an unfamiliar pattern, and thus characterizing it as a fault,
- Convolutional LSTM (ConvLSTM) for all the stages of the FLITC application, i.e., FFNN, FBNN, FCNN, and FDNN [57]. We leverage ConvLSTM, which can consider the spatiotemporal characteristics of the data and outperform MLP [58]. In this work, the data stemming from the measuring devices inherently contain the grid's topology characteristics.

5.3. Performance evaluation

For the evaluation of the model, a set of metrics is used to elucidate each layer's performance from different perspectives. Firstly, the confusion matrix is utilized for the case of a multi-class classification DNN model. Specifically, it depicts a matrix of $n \times n$ dimensions, where n denotes the number of classes the DNN is trained for. Across the matrix rows lie the true labels of the test data, while across the columns are the predicted ones. The diagonal of the matrix showcases the correct classifications of the DNN models. For binary confusion matrices, where there are only two classes to select from, each value of the matrix belongs to one of the following categories:

- True positives (TP): an outcome where the model correctly predicts the positive class,
- True negatives (TN): an outcome where the model correctly predicts the negative class,
- False positives (FP): an outcome where the model incorrectly predicts the positive class,
- False negatives (FN): an outcome where the model incorrectly predicts the negative class.

Beyond the confusion matrix of the model, the accuracy, precision, recall, and F1-score of the DNN are calculated. Accuracy is the fraction of all correctly classified values over the entire dataset. On the other hand, precision is defined as the fraction for a particular class of the correctly classified values by the total number of predicted ones. Recall is the fraction of correctly predicted values by all the true labels. Lastly, F1-score is the harmonic mean of the precision and recall. The formulas of the above-described metrics are:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \tag{12}$$

$$Precision = \frac{TP}{TP + FP} \tag{13}$$

$$Recall = \frac{TP}{TP + FN} \tag{14}$$

$$F1 - score = 2\frac{Precision \times Recall}{Precision + Recall} \tag{15}$$

For the regression problem, the Root Mean Square Error (RMSE) metric is used, i.e., a quadratic scoring rule that calculates the average magnitude of the error. It is the square root of the average of squared differences between prediction and actual observation given by the formula:

$$RMSE = \sqrt{\frac{\sum_{i=1}^{N}(x_i - x_i')^2}{N}} \tag{16}$$

where i are the different training samples, N is the amount of training data points, x_i is the actual fault distance and x' is the estimated by model distance. Since the errors are squared before they are averaged, the RMSE gives a relatively high weight to large errors. This means the RMSE should be more useful when large errors are particularly undesirable. This applies directly to the FLITC case, due to the fact that its primary objective is to reduce the time that the operators' inspection teams spend to find the exact fault location.

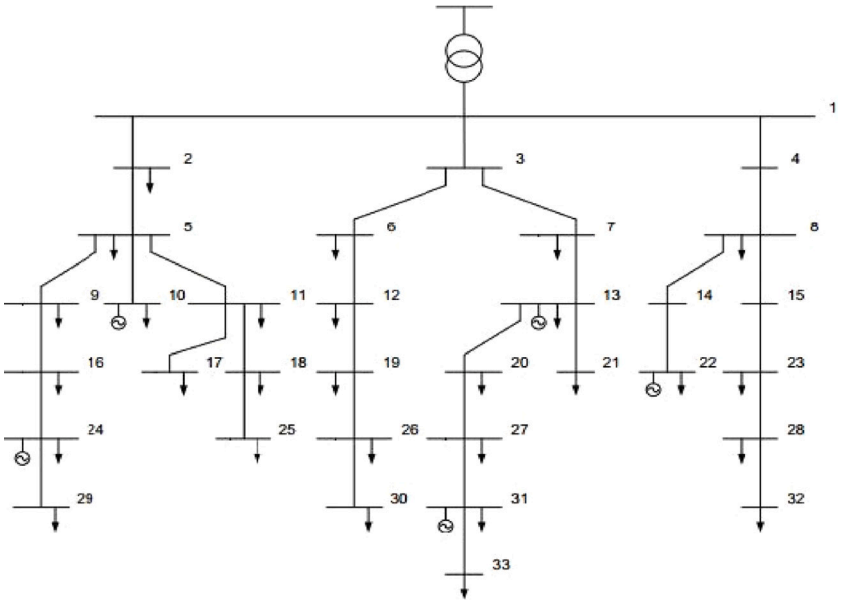


Fig. 6. Topology of the LVDG used as a testbed.

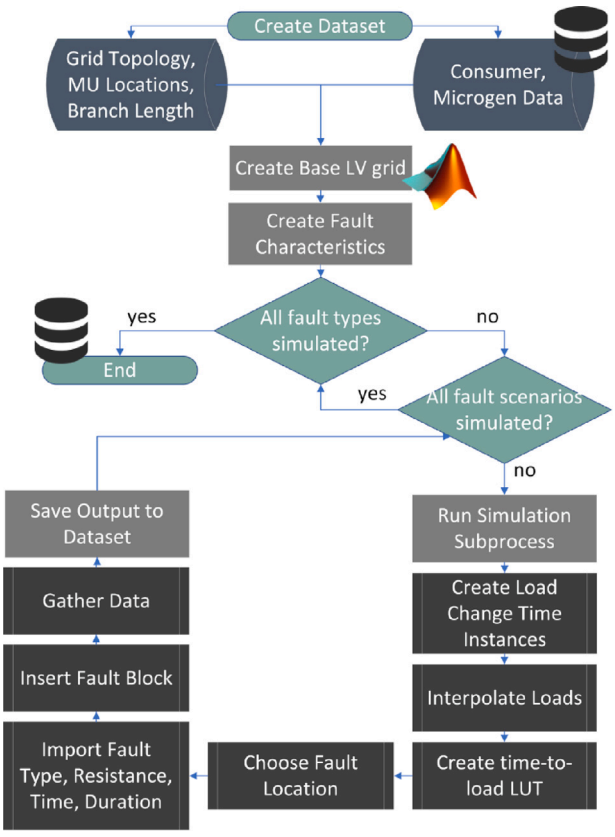


Fig. 7. Flowchart of the dataset generation process.

5.4. Computational environment

For the dataset generation process, the simulation model is developed in the Matlab-based Simulink graphical programming environment. The FLITC application models are developed in python, by using the TensorFlow library with Keras as an API. For hyperparameter tuning, the HyperOpt, i.e., an open-source library for large-scale automated machine learning, is used [59]. HyperOpt gives the capability of conducting hyperparameter tuning through the utilization of the TPE algorithm. All the computations were conducted on an AlmaLinux server with 8 CPU cores with a total of 16 GB RAM and an NVIDIA RTX A6000 GPU card with a total RAM capacity equal to 48 GB.

6. Results

6.1. FLITC performance

Table 2 includes the best models for each stage of the FLITC application, as derived from the hyperparameter tuning process. For all the models, padding is set to "same" and not "valid", meaning that we keep the dimensionality of the input vector for stride equal to 1, and Maxpooling is set equal to 2*2 size, meaning that we reduce the size of each feature map by a factor of 2. The validation accuracy for the best models is 90.76% for the FFNN, 85.4%, 93.77%, 95.02% for each feeder model, namely FBNN-1, FBNN-2, and FBNN-3, respectively, and 94.93% for the FCNN. The RMSE value is equal to 2.45% for the FDNN model. The most computationally intensive derived model is the FCNN, which consists of 4 hidden layers and has a mean training time for each epoch equal to 9.6 s.

Fig. 9 illustrates the confusion matrices for the best three FFNN models derived from the hyperparameter process. As it can be thoroughly observed, even though the model with the highest accuracy is the one in the upper left, it exhibits worse performance in detecting the healthy states compared to the lower depicted model (precision value for healthy state 84.97% and 90.62%, respectively). For the fault diagnosis process, it is of crucial importance the distinction between the healthy and faulty states. Therefore, the sensitivity of the FLITC application, i.e., the recall metric for binary classification, to identify the faulty occurrences is equal to 91.4% and the specificity is equal to 90.62%. In addition, as illustrated in Fig. 10, the performance of the FFNN model is identical regardless of the fault duration. On the other hand, Fig. 11 depicts that the fault diagnosis performance significantly decreases, as long as the fault resistance increases. This is mainly caused due to the inability of the models to capture the high resistance faults, which produce smaller variations to the magnitude of the grid variables and have less cascading effects on neighboring branches and feeders.

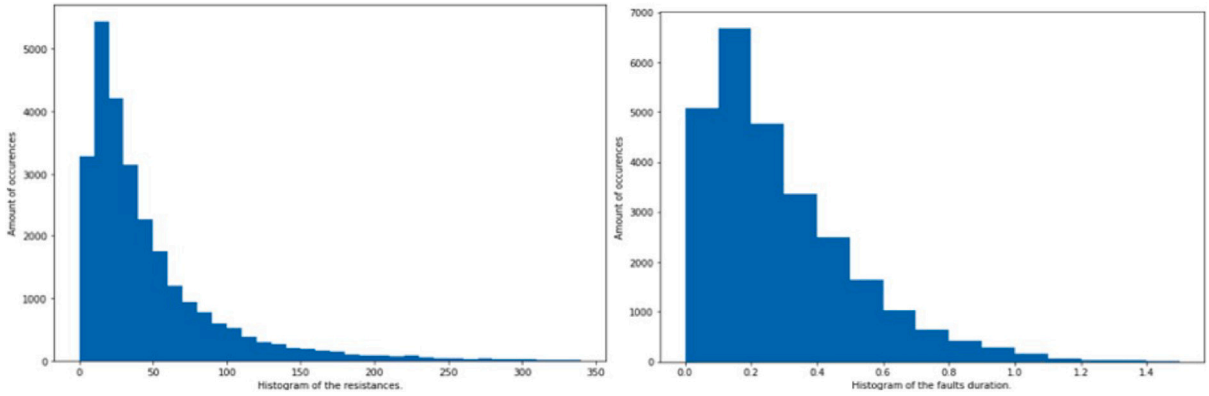


Fig. 8. Histograms of resistance and duration values of the generated faults.

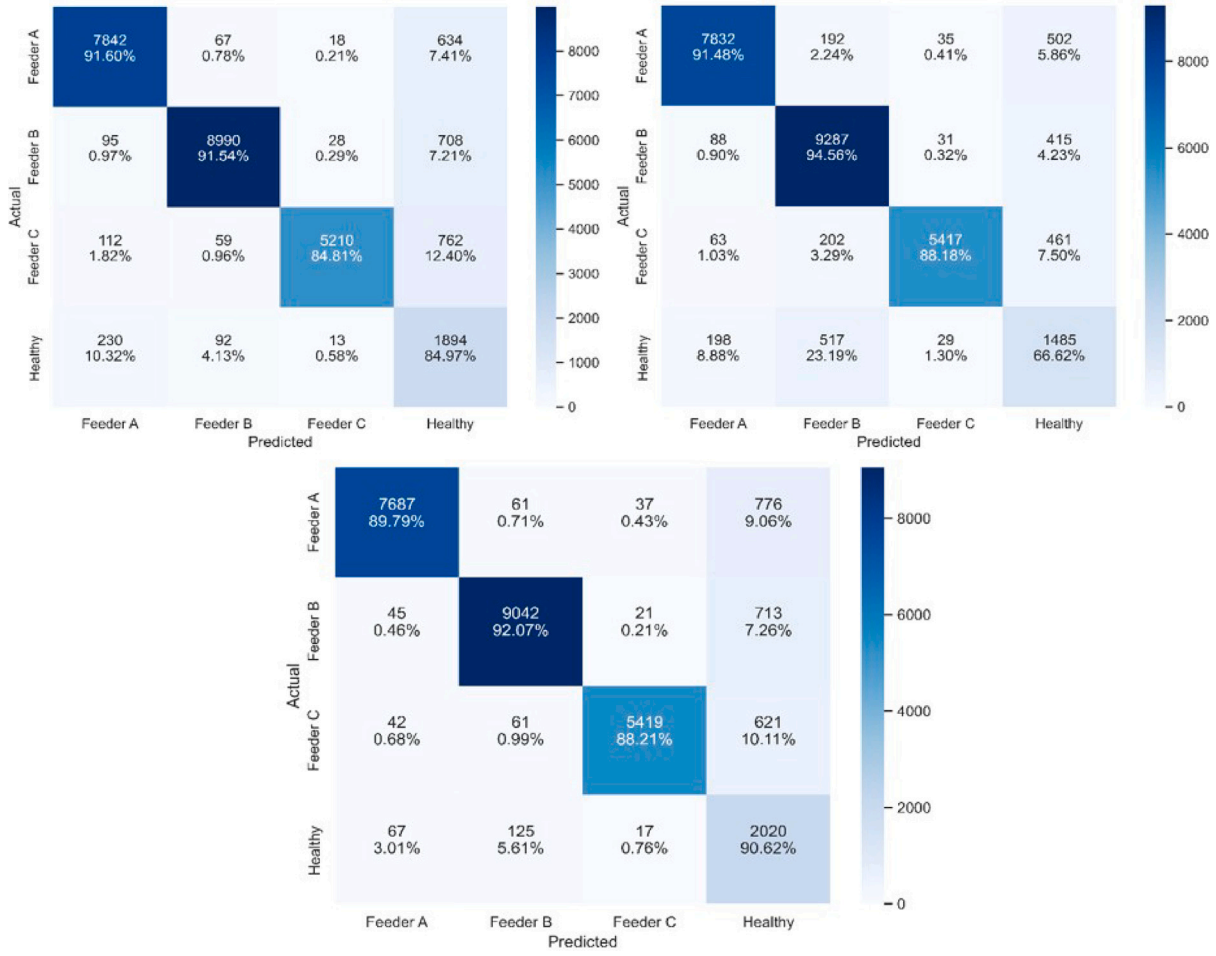


Fig. 9. Confusion matrices for the best FFNN models. The upper left is the best model with a total accuracy of 90.5%. The upper right is the second best model with accuracy equal to 89.32%, and the lower one is the third best model with accuracy equal to 88.74%.

Fig. 12 illustrates the confusion matrix for the best FBNN model concerning the feeder with the four branches. The low accuracy of that model compared to the rest of the FBNN models is mainly attributed to its inability to classify the faults of the second branch correctly due to its small length (the existence of only one node). This can be considered as a limitation of the model. Other models derived from the hyperparameter tuning process have better accuracy in detecting the faults for that particular branch, however, their overall accuracy was lower.

Fig. 13 presents the confusion matrix for the FCNN model. The 2Φ and 3Φ to ground faults are grouped in the same category as the ones without ground. As it can be seen, the model exhibits almost flawless performance in classifying the 2Φ and 3Φ faults, except for the A-B fault, where all the misclassified faults are categorized as 1Φ B to ground faults, probably due to the network asymmetry and the fact that 1Φ B has the most connected loads. For the 1Φ faults, the accuracy of the FCNN model drops due to the misclassification of the faults to the rest 1Φ fault categories.

Table 2
Derivation of the best models for the FLITC application from the hyperparameter tuning process.

	FFNN	FBNN-1	FBNN-2	FBNN-3	FCNN	FDNN
Hidden Layers	2	1	1	2	4	3
Hidden Layer Units	[128, 160]	224	192	[80, 240]	[144, 80, 144, 96]	[192, 48, 208]
Regularization	Dropout	Dropout, L2	Dropout, L2	Dropout	Dropout, L2	Dropout
Dropout Rate	[0.05, 0.025, 0.0167, 0.0125]	[0.05, 0.025, 0.0167]	[0.05, 0.025, 0.0167]	[0.05, 0.025, 0.0167, 0.0125]	[0.05, 0.025, 0.0167, 0.0125, 0.01, 0.0083]	[0.05, 0.025, 0.0167, 0.0125, 0.01]
L2 Regularization Factor	–	0.01	0.01	–	0.01	–
Accuracy (%)	90.49	85.4	93.77	95.02	94.93	–
Loss (%)	29.62	52.7	26.92	15.13	21.37	0.12
F1-score (%)	62.36 - 93.90	16.22 - 88.78	83.75 - 95.56	94.91 - 95.25	89.87 - 99.10	–
RMSE (%)	–	–	–	–	–	2.45
Mean epoch time (s)	1.82	1.24	3.7	0.73	9.6	2.52

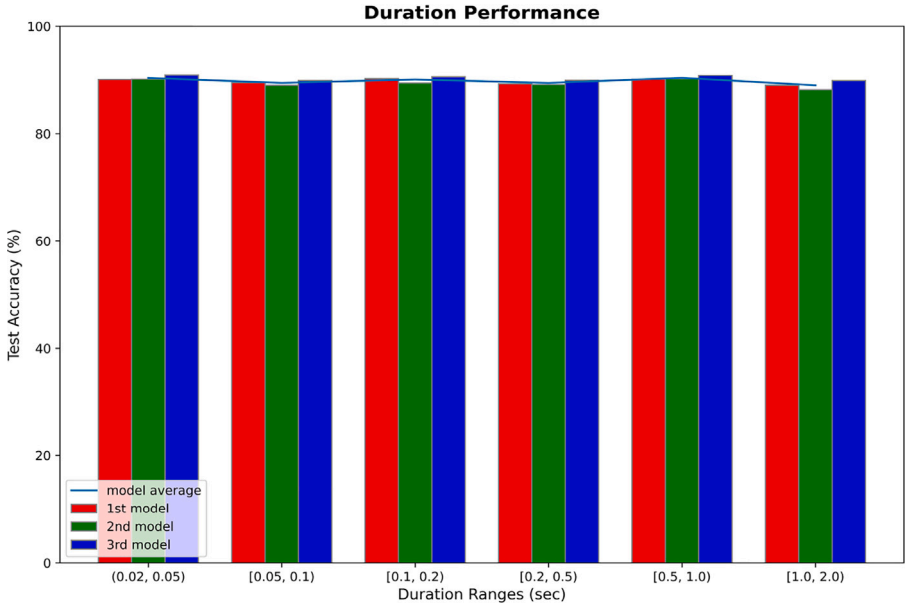


Fig. 10. Average test performance for the different fault duration ranges of the best FFNN models.

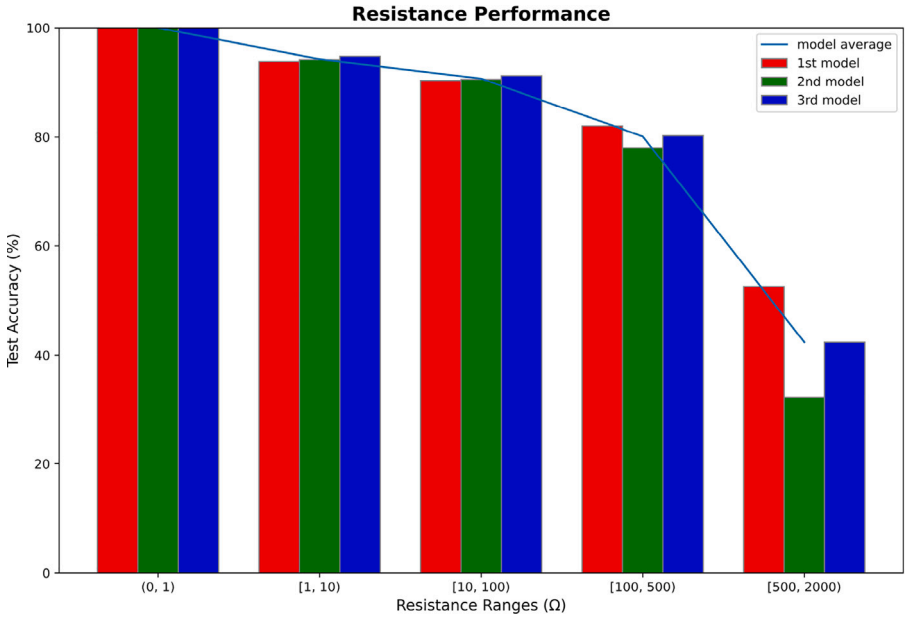


Fig. 11. Average test performance for the different classes of fault resistance of the best FFNN models.

Fig. 14 illustrates the density function of the absolute error, normalized by the maximum distance, as calculated from the FDNN model. It can be seen that the computed normalized error values loosely follow the beta distribution. The blue line represents the beta distribution

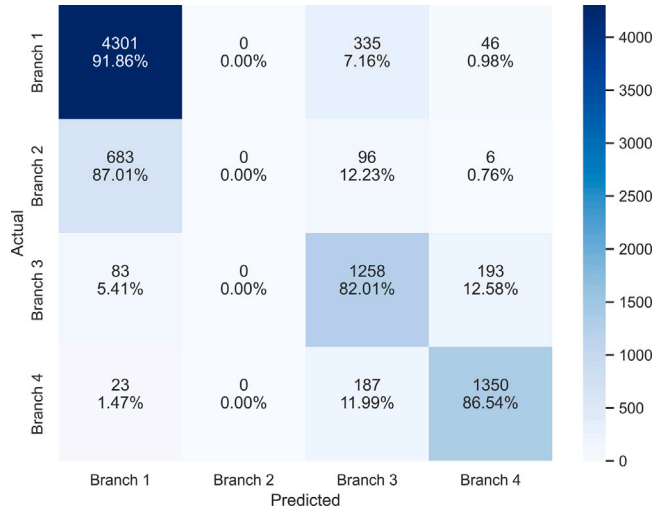


Fig. 12. Confusion matrix of the FBNN model for the first feeder.

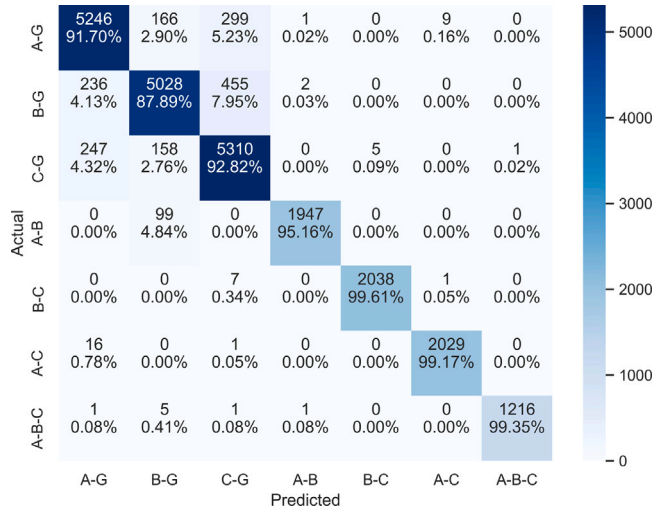


Fig. 13. Confusion matrix for the best FCNN model.

with parameters $a = 0.52$ and $b = 329.77$, which have the best goodness-of-fit ratio, i.e., the lowest Residual Sum of Squares (RSS) value (95.52), among other 89 univariate distributions using the library from [60]. The calculated fault distances from the FDNN model are below the 7.21% of the normalized branch distance from the root node with a 95% confidence. Therefore, we can conclude that the FDNN model exhibits high performance providing valuable assistance to the maintenance crew of the operators to locate the fault correctly.

Regarding the execution times of data preprocessing and model inference stages in the different modules we obtained: 173 ms for the FFNN, for the FBNN-1,-2, –3 479 ms, 413 ms, and 435 ms, respectively, 278 ms for the FCNN, and 119 ms for the FDNN.

6.2. Comparison to benchmark models

We conduct evaluation of the proposed FLITC application, compared to other benchmark models, summarized below:

- Fault detection: A comparison takes place between the FLITC application's ability to detect the faulty states, in comparison with an unsupervised method that is used broadly across the academia for fault diagnosis and anomaly detection, namely the LSTM Autoencoder. Our proposed model exhibits slightly worse sensitivity than the LSTM Autoencoder in detecting the faulty states (91.6% compared to 91.4%),
- Feeder and branch identification: For the faulty feeder detection (if fault exists) the best hyperparameter optimally tuned ConvLSTM model (batch size = 200, units = 180, dropout rate = 0.2, hidden layers = 4, window size = 50, and filter size = 256) achieves an accuracy of 87.12%, which is lower than the 90.49% achieved by the FFNN model of the FLITC application. For the branch identification, our model outperforms all the ConvLSTM models in all the three different feeders, i.e., 85.4% > 67.43%, 93.76% > 74.65%, and 95.02% > 76.42%.
- Faulty class: Our model outperforms the best hyperparameter optimally tuned ConvLSTM model, which is used as multi-class classifier. Particularly, the best accuracy achieved by the ConvLSTM (batch size = 150, units = 160, dropout rate = 0.2, hidden layers = 3, window size = 25 and filters = 64) is equal to 83.1%, which is significantly lower than the 94.93% achieved by the FLITC application.
- Distance calculation: Our model outperforms the best hyperparameter optimally tuned ConvLSTM model, which is used as a regressor. Particularly, the lowest RMSE value achieved by the ConvLSTM (batch size = 200, units = 80, dropout rate = 0.2, hidden layers = 2, window size = 25 and filters = 64) is equal to 21.54%, i.e., significantly lower than the 2.52% attained by the FLITC application.

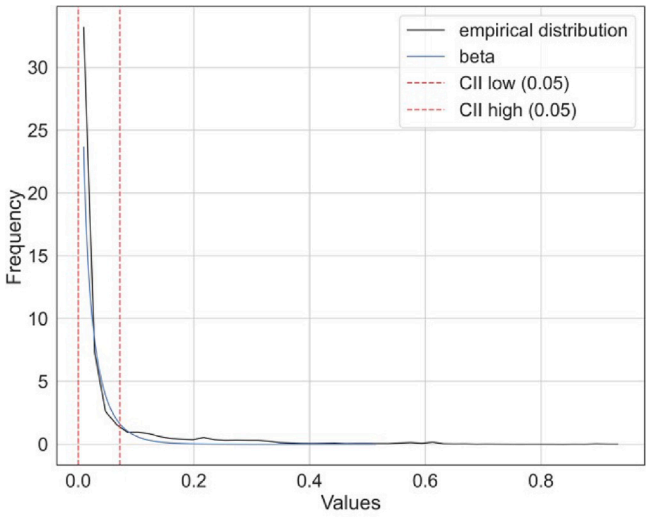


Fig. 14. Density function of the observed error values calculated from the FDNN model.

6.3. Sensitivity analysis

To explore the robustness of the FLITC application, we conduct a thorough analysis to quantify the performance of the best models in LVDGs against different availability rates of measuring units. For a particular network topology, we define the unit availability rate as the fraction of nodes in which $3\Phi - V$ measuring units exist, divided by the total number of nodes. We consider that devices measuring $3\Phi - I$ in the root of each feeder and $3\Phi - V$ at the first and last node of each branch exist in all the cases. This sensitivity study is essential for distribution system operators' planning and innovation strategies since it provides the minimum requirements for infrastructure upgrades so as to integrate automatic fault diagnosis and self-healing practices.

In the results presented in Section 6.1, we quantified the performance of the FLITC application for an availability rate of 100%. Fig. 15 showcases the mean performance of the models (except for the FBNN, which uses $3\Phi - I$ values) for measuring units availability rate ranging

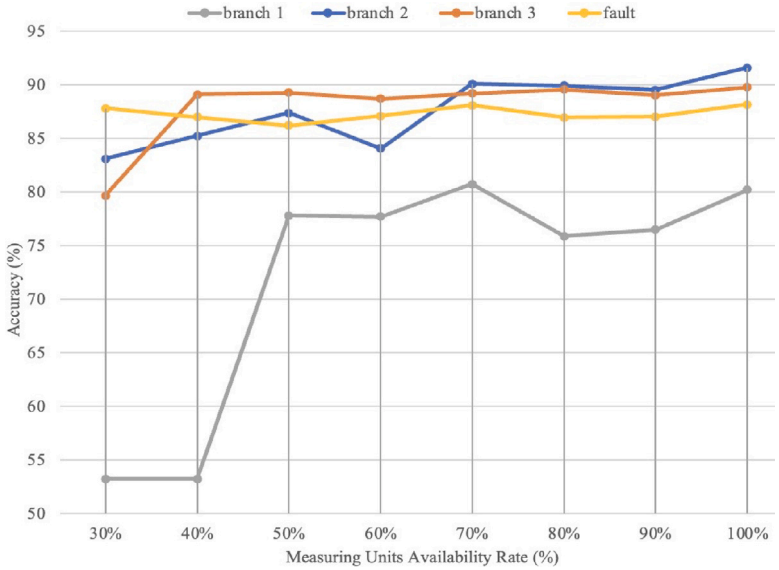


Fig. 15. Testing accuracy of the FBNN and FCNN models for different levels of measuring devices availability rate.

from 30% to 100% (we train each model 100 times and compute the mean performance for the test dataset). As it can be seen, there is a trend in most of the models to have better accuracy with higher availability rates. However, since we use the interpolation step in the preprocessing stage, we can conclude that there is no significant drop in the models' accuracy if the availability rate is higher than 50% and adequate results could be obtained. For the FDNN model, the normalized error limit for the 95% confidence interval reaches up to 23% for measuring availability rate of 30%, compared to the 7.21% for 100%.

6.4. Application limitations

The DNN-based approach followed in this work, has three limitations that are discussed in this section. First, it requires a large amount of data for their training, which are often not available, especially when it comes to real data. To the authors' knowledge, there is no publicly available dataset for fault diagnosis studies in LVDGs, due to the fact that structural and infrastructure upgrades have taken place in LVDGs over the last years. Thus, obtaining curated high-resolution datasets from a real operational environment is not yet feasible. Since real data are unavailable, our method is based on synthetic data, which in turn requires a deep knowledge of the grid parameters to approximate faulty conditions, as accurately as possible. In Section 5, we have thoroughly presented the analysis of the data generation process while making publicly available the source code. Second, due to the supervised nature of the FLITC application, the models' accuracy is guaranteed neither in fault cases that are not included in the training phase nor when changes in the LVDGs topology/measuring infrastructure occur. The preprocessing stage employed in this work, particularly the interpolation and dimensionality reduction techniques, increases the generalizability of the model and its performance robustness, with the cost of increasing also the complexity and the computational time of the models. Last, the authors recognize the limitations of the model to detect faults in mesh topologies that night exist even at the edge of the LVDGs.

7. Conclusions

The primary focus of this paper is the selection of efficient data-driven DNN-based methods to create an application that is able to be used as a fault diagnostic tool for smart LVDGs. Cornerstones of the proposed FLITC application are the CWT and CNNs models, which are able to handle the big amount of data stemming from the measuring units and detect with high accuracy the fault patterns. Furthermore, by using the TPE algorithm for conducting exploration of the most suitable hyperparameters for a specific aspect of the fault diagnosis tool, i.e., the faulty feeder, branch, class, and distance models. The results showcased its efficacy in providing fine-grained and accurate fault diagnosis analytics to the system operators, i.e., an accuracy of 91.4% for fault detection, 93.77% for correct branch identification, 94.93% for fault type classification, and RMSE value of 2.45% for location calculation. Further work could be conducted to make the FLITC application applicable in mesh network topologies and explore both privacy-preserving and resource-constraint methods for fault diagnosis in LVDG leveraging the computation resources at the edge.

CRediT authorship contribution statement

V. Rizeakos: Conceptualization, Methodology/Study design, Software, Validation, Formal analysis, Investigation, Data curation, Writing – original draft, Visualization. **A. Bachoumis:** Conceptualization, Methodology/Study design, Software, Validation, Formal analysis, Investigation, Writing – original draft, Writing – review & editing, Supervision. **N. Andriopoulos:** Formal analysis, Writing – original draft, Writing – review & editing, Supervision. **M. Birbas:** Resources, Writing – review & editing, Supervision, Project administration, Funding acquisition. **A. Birbas:** Resources, Writing – review & editing, Supervision, Project administration, Funding acquisition.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: Athanasios Bachoumis reports financial support was provided by EU Framework Programme for Research and Innovation Societal Challenges.

Data availability

We have shared the link for the open source repo that this work's used data and code have been uploaded.

References

[1] Saldaña-González AE, Sumper A, Aragüés-Peñalba M, Smolnikar M. Advanced distribution measurement technologies and data applications for smart grids: A review. Energies 2020;13(14). http://dx.doi.org/10.3390/en13143730, URL https://www.mdpi.com/1996-1073/13/14/3730.

[2] Barja-Martinez S, Aragüés-Peñalba M, Munné-Collado Í, Lloret-Gallego P, Bullich-Massagué E, Villafafila-Robles R. Artificial intelligence techniques for enabling big data services in distribution networks: A review. Renew Sustain Energy Rev 2021;150:111459.

[3] Cipcigan L, Taylor P. Investigation of the reverse power flow requirements of high penetrations of small-scale embedded generation. IET Renew Power Gener 2007;1(3):160–6.

[4] Liserre M, Sauter T, Hung JY. Future energy systems: Integrating renewable energy sources into the smart power grid through industrial electronics. IEEE Ind Electron Mag 2010;4(1):18–37.

[5] Shittu E, Tibrewala A, Kalla S, Wang X. Meta-analysis of the strategies for self-healing and resilience in power systems. Adv Appl Energy 2021;100036.

[6] Arefifar SA, Mohamed YA-RI, El-Fouly TH. Comprehensive operational planning framework for self-healing control actions in smart distribution grids. IEEE Trans Power Syst 2013;28(4):4192–200.

[7] Shirazi E, Jadid S. Autonomous self-healing in smart distribution grids using agent systems. IEEE Trans Ind Inf 2018;15(12):6291–301.

[8] Ramos M, Resener M, Oliveira P, Bernardon DP. Self-healing in power distribution systems. In: Smart operation for power distribution systems. Springer; 2018, p. 37–70.

[9] Teng J-H. A direct approach for distribution system load flow solutions. IEEE Trans Power Deliv 2003;18(3):882–7.

[10] Mora-Florez J, Meléndez J, Carrillo-Caicedo G. Comparison of impedance based fault location methods for power distribution systems. Electr Power Syst Res 2008;78(4):657–66.

[11] Salim R, Salim K, Bretas A. Further improvements on impedance-based fault location for power distribution systems. IET Gener Transm Distrib 2011;5(4):467–78.

[12] Aboshady F, Thomas D, Sumner M. A new single end wideband impedance based fault location scheme for distribution systems. Electr Power Syst Res 2019;173:263–70.

[13] Ayambire PN, Huang Q, Cai D, Bamisile O, Anane POK. Real-time and contactless initial current traveling wave measurement for overhead transmission line fault detection based on tunnel magnetoresistive sensors. Electr Power Syst Res 2020;187:106508.

[14] Aftab MA, Hussain SS, Ali I, Ustun TS. Dynamic protection of power systems with high penetration of renewables: A review of the traveling wave based fault location techniques. Int J Electr Power Energy Syst 2020;114:105410.

[15] Zidan A, Khairalla M, Abdrabou AM, Khalifa T, Shaban K, Abdrabou A, et al. Fault detection, isolation, and service restoration in distribution systems: State-of-the-art and future trends. IEEE Trans Smart Grid 2016;8(5):2170–85.

[16] Bahmanyar A, Jamali S, Estebsari A, Bompard E. A comparison framework for distribution system outage and fault location methods. Electr Power Syst Res 2017;145:19–34.

[17] Agrawal R, Thukaram D. Identification of fault location in power distribution system with distributed generation using support vector machines. In: 2013 IEEE PES innovative smart grid technologies conference. IEEE; 2013, p. 1–6.

[18] Fei W, Moses P. Fault current tracing and identification via machine learning considering distributed energy resources in distribution networks. Energies 2019;12(22):4333.

[19] Sapountzoglou N, Lago J, Raison B. Fault diagnosis in low voltage smart distribution grids using gradient boosting trees. Electr Power Syst Res 2020;182:106254.

[20] Yadav A, Swetapadma A. A novel transmission line relaying scheme for fault detection and classification using wavelet transform and linear discriminant analysis. Ain Shams Eng J 2015;6(1):199–209.

[21] Perez F, Orduna E, Guidi G. Adaptive wavelets applied to fault classification on transmission lines. IET Gener Transm Distrib 2011;5(7):694–702.

[22] Shafiullah M, Abido MA, Al-Hamouz Z. Wavelet-based extreme learning machine for distribution grid fault location. IET Gener Transm Distrib 2017;11(17):4256–63.

[23] Jiang H, Zhang JJ, Gao W, Wu Z. Fault detection, identification, and location in smart grid based on data-driven computational methods. IEEE Trans Smart Grid 2014;5(6):2947–56.

[24] Tervo R, Karjalainen J, Jung A. Predicting electricity outages caused by convective storms. In: 2018 IEEE data science workshop. IEEE; 2018, p. 145–9.

[25] Souto L, Meléndez J, Herraiz S. Fault location in low voltage smart grids based on similarity criteria in the principal component subspace. In: 2020 IEEE power & energy society innovative smart grid technologies conference. IEEE; 2020, p. 1–5.

[26] Chakraborty D, Sur U, Banerjee PK. Random forest based fault classification technique for active power system networks. In: 2019 IEEE international WIE conference on electrical and computer engineering. IEEE; 2019, p. 1–4.

[27] Mamuya YD, Lee Y-D, Shen J-W, Shafiullah M, Kuo C-C. Application of machine learning for fault classification and location in a radial distribution grid. Appl Sci 2020;10(14):4965.

[28] Sapountzoglou N, Lago J, De Schutter B, Raison B. A generalizable and sensor-independent deep learning method for fault detection and location in low-voltage distribution grids. Appl Energy 2020;276:115299.

[29] Rai P, Londhe ND, Raj R. Fault classification in power system distribution network integrated with distributed generators using CNN. Electr Power Syst Res 2021;192:106914.

[30] Li W, Deka D, Chertkov M, Wang M. Real-time faulted line localization and PMU placement in power systems through convolutional neural networks. IEEE Trans Power Syst 2019;34(6):4640–51.

[31] Salim RH, de Oliveira KRC, Filomena AD, Resener M, Bretas AS. Hybrid fault diagnosis scheme implementation for power distribution systems automation. IEEE Trans Power Deliv 2008;23(4):1846–56.

[32] Galijasevic Z, Abur A. Fault location using voltage measurements. IEEE Trans Power Deliv 2002;17(2):441–5.

[33] Stefanidou-Voziki P, Sapountzoglou N, Raison B, Dominguez-Garcia J. A review of fault location and classification methods in distribution grids. Electr Power Syst Res 2022;209:108031.

[34] Falaghi H, Haghifam M-R, Tabrizi MO. Fault indicators effects on distribution reliability indices. In: CIRED 2005-18th international conference and exhibition on electricity distribution. IET; 2005, p. 1–4.

[35] Vasilis Rizeakos VR, Athanasios Bachoumis AB. FLITC-application, URL https://github.com/tombax7/FLITC-application.

[36] Mar A, Pereira P, F. Martins J. A survey on power grid faults and their origins: A contribution to improving power grid resilience. Energies 2019;12(24). http://dx.doi.org/10.3390/en12244667, URL https://www.mdpi.com/1996-1073/12/24/4667.

[37] Hare J, Shi X, Gupta S, Bazzi A. Fault diagnostics in smart micro-grids: A survey. Renew Sustain Energy Rev 2016;60:1114–24. http://dx.doi.org/10.1016/j.rser.2016.01.122.

[38] Taft J. Fault intelligence: Distribution grid fault detection and classification. Washington, DC, USA: Pacific Northwest National Laboratory, Richland; Department of Energy; 2017, p. 99352.

[39] Grainger JJ. Power system analysis. McGraw-Hill; 1999.

[40] Schmid PJ. Dynamic mode decomposition of numerical and experimental data. J Fluid Mech 2010;656:5–28.

[41] Meintjes A, Lowe A, Legget M. Fundamental heart sound classification using the continuous wavelet transform and convolutional neural networks. In: 2018 40th annual international conference of the IEEE engineering in medicine and biology society. IEEE; 2018, p. 409–12.

[42] Miao R, Gao Y, Ge L, Jiang Z, Zhang J. Online defect recognition of narrow overlap weld based on two-stage recognition model combining continuous wavelet transform and convolutional neural network. Comput Ind 2019;112:103115.

[43] Nguyen H-P, Liu J, Zio E. A long-term prediction approach based on long short-term memory neural networks with automatic parameter optimization by tree-structured parzen estimator and applied to time-series data of NPP steam generators. Appl Soft Comput 2020;89:106116.

[44] Bergstra J, Bardenet R, Bengio Y, Kégl B. Algorithms for hyper-parameter optimization. Adv Neural Inf Process Syst 2011;24.

[45] Rodrigues LF, Naldi MC, Mari JF. Comparing convolutional neural networks and preprocessing techniques for HEp-2 cell classification in immunofluorescence images. Comput Biol Med 2020;116:103542.

[46] Chevtchenko SF, Vale RF, Macario V, Cordeiro FR. A convolutional neural network with feature fusion for real-time hand posture recognition. Appl Soft Comput 2018;73:748–66.

[47] Lago J, De Ridder F, Vrancx P, De Schutter B. Forecasting day-ahead electricity prices in Europe: the importance of considering market integration. Appl Energy 2018;211:890–903.

[48] He F, Zhou J, Mo L, Feng K, Liu G, He Z. Day-ahead short-term load probability density forecasting method with a decomposition-based quantile regression forest. Appl Energy 2020;262:114396.

[49] Fekri MN, Patel H, Grolinger K, Sharma V. Deep learning for load forecasting with smart meter data: Online adaptive recurrent neural network. Appl Energy 2021;282:116177.

[50] Lago J, De Brabandere K, De Ridder F, De Schutter B. Short-term forecasting of solar irradiance without local telemetry: A generalized model using satellite data. Sol Energy 2018;173:566–77.

[51] Dataport. Pecan Street Inc.; 2020, URL https://www.pecanstreet.org/dataport/.

[52] Sapountzoglou N. Detection et localisation des defauts dans les resaux de distribution basse tension en presence de production decentralisee [Ph.D. thesis], Université Grenoble Alpes (ComUE); 2019.

[53] Musa H. A review of distributed generation resource types and their mathematical models for power flow analysis. Int J Sci Technol Soc 2015;3(4):204.

[54] Katiraei F, Holbach J, Chang T, Johnson W, Wills D, Young B, et al. Investigation of solar PV inverters current contributions during faults on distribution and transmission systems interruption capacity. In: Western protective relay conference. 2012, p. 70–8.

[55] Nguyen H, Tran KP, Thomassey S, Hamad M. Forecasting and anomaly detection approaches using LSTM and LSTM autoencoder techniques with the applications in supply chain management. Int J Inf Manage 2021;57:102282.
[56] Said Elsayed M, Le-Khac N-A, Dev S, Jurcut AD. Network anomaly detection using LSTM based autoencoder. In: Proceedings of the 16th ACM symposium on qos and security for wireless and mobile networks. 2020, p. 37–45.
[57] Rizeakos V, Bachoumis A, Birbas A, Birbas M. Convlstm-based fault detection and classification in active distribution grids. In: 2022 International conference on electrical, computer and energy technologies. IEEE; 2022, p. 1–6.
[58] Luo W, Liu W, Gao S. Remembering history with convolutional lstm for anomaly detection. In: 2017 IEEE international conference on multimedia and expo. IEEE; 2017, p. 439–44.
[59] Bergstra J, Yamins D, Cox D. Making a science of model search: Hyperparameter optimization in hundreds of dimensions for vision architectures. In: International conference on machine learning. PMLR; 2013, p. 115–23.
[60] Taskesen E. distfit - Probability density fitting. 2020, URL https://erdogant.github.io/distfit.